

Module 6: Arnav Kucheriya

Saturday, February 28, 2026 11:06 AM

Available (A,B,C,D) = (3, 3, 2, 1)

Maximum and Allocation

Process	Max (A B C D)	Alloc (A B C D)
P1	(4,2,1,2)	(2,0,0,1)
P2	(5,2,5,2)	(3,1,2,1)
P3	(2,3,1,6)	(2,1,0,3)
P4	(1,4,2,4)	(1,3,1,2)
P5	(3,6,6,5)	(1,4,3,2)
P6	(4,2,3,5)	(1,0,2,2)

A) Compute the Need Matrix

Need = Maximum – Allocation

Need (A,B,C,D)

- P1: $(4,2,1,2) - (2,0,0,1) = (2,2,1,1)$
- P2: $(5,2,5,2) - (3,1,2,1) = (2,1,3,1)$
- P3: $(2,3,1,6) - (2,1,0,3) = (0,2,1,3)$
- P4: $(1,4,2,4) - (1,3,1,2) = (0,1,1,2)$
- P5: $(3,6,6,5) - (1,4,3,2) = (2,2,3,3)$
- P6: $(4,2,3,5) - (1,0,2,2) = (3,2,1,3)$

Need Matrix:

Process	Need (A B C D)
P1	(2,2,1,1)
P2	(2,1,3,1)
P3	(0,2,1,3)
P4	(0,1,1,2)
P5	(2,2,3,3)
P6	(3,2,1,3)

B) Is the system in a safe state? If yes, show a safe sequence (all steps)

Step 0: Initialize

- Work = Available = (3,3,2,1)
- Finish = [F,F,F,F,F,F] (for P1..P6)

Process with Need $\leq$ Work.

Step 1: Check processes against Work = (3,3,2,1)

P1 Need (2,2,1,1) $\leq$ (3,3,2,1) can run.

After P1 finishes, it releases its Allocation (2,0,0,1).

Work = (3,3,2,1) + (2,0,0,1) = (5,3,2,2)

Finish: P1 = T

Safe sequence: [P1]

Step 2: Work = (5,3,2,2)

P4 Need (0,1,1,2) $\leq$ (5,3,2,2) can run.

Release P4 Allocation (1,3,1,2)

Work = (5,3,2,2) + (1,3,1,2) = (6,6,3,4)

Finish: P4 = T

Sequence: [P1, P4]

Step 3: Work = (6,6,3,4)

P5 Need (2,2,3,3) $\leq$ (6,6,3,4) can run.

Release P5 Allocation (1,4,3,2)

Work = (6,6,3,4) + (1,4,3,2) = (7,10,6,6)

Finish: P5 = T

Sequence: [P1, P4, P5]

Step 4: Work = (7,10,6,6)

P6 Need (3,2,1,3) $\leq$ (7,10,6,6) can run.

Release P6 Allocation (1,0,2,2)

Work = (7,10,6,6) + (1,0,2,2) = (8,10,8,8)

Finish: P6 = T

Sequence: [P1, P4, P5, P6]

Step 5: Work = (8,10,8,8)

P2 Need (2,1,3,1) $\leq$ (8,10,8,8) can run.

Release P2 Allocation (3,1,2,1)

Work = (8,10,8,8) + (3,1,2,1) = (11,11,10,9)

Finish: P2 = T

Sequence: [P1, P4, P5, P6, P2]

Step 6: Work = (11,11,10,9)

P3 Need (0,2,1,3) $\leq$ (11,11,10,9) can run.

Release P3 Allocation (2,1,0,3)

Work = (11,11,10,9) + (2,1,0,3) = (13,12,10,12)

Finish: P3 = T

Sequence: [P1, P4, P5, P6, P2, P3]

Conclusion (Safe State)

All processes can finish (all Finish = True), so the system is in a safe state.

One valid safe sequence is:

$\langle P1, P4, P5, P6, P2, P3 \rangle$

C) If P2 requests (1,1,0,1), should it be granted immediately?

Request(P2) = (1,1,0,1)

Step C1: Check Request $\leq$ Need(P2)

Need(P2) = (2,1,3,1)

Is (1,1,0,1) $\leq$ (2,1,3,1)?

A: $1 \leq 2$

B: $1 \leq 1$

C: $0 \leq 3$

D: $1 \leq 1$

Yes, request is within P2's remaining need.

Step C2: Check Request $\leq$ Available

Available = (3,3,2,1)

Is (1,1,0,1) $\leq$ (3,3,2,1)?

A: $1 \leq 3$

B: $1 \leq 3$

C: $0 \leq 2$

D: $1 \leq 1$

Yes, resources are available right now.

Step C3: Pretend to allocate (Banker's test)

If we grant the request:

Available' = Available – Request

= (3,3,2,1) – (1,1,0,1) = (2,2,2,0)

Allocation(P2)' = Allocation(P2) + Request

= (3,1,2,1) + (1,1,0,1) = (4,2,2,2)

Need(P2)' = Need(P2) – Request

= (2,1,3,1) – (1,1,0,1) = (1,0,3,0)

Now check if the system is still safe with Work = (2,2,2,0).

Step C4: Safety check with Work = (2,2,2,0)

Check each process Need $\leq$ Work:

P1 Need (2,2,1,1): fails because D = 1 > 0

P2 Need' (1,0,3,0): fails because C = 3 > 2

P3 Need (0,2,1,3): fails because D = 3 > 0

P4 Need (0,1,1,2): fails because D = 2 > 0

P5 Need (2,2,3,3): fails because D = 3 > 0 and C = 3 > 2

P6 Need (3,2,1,3): fails because A = 3 > 2 and D = 3 > 0

No process can run first. That means the system would be unsafe if we grant it immediately.

Conclusion for (C)

Do not grant P2's request immediately, because after granting it, the resulting state has no valid starting process, so it is not safe under Banker's Algorithm.